

current job, you can use `C-c C-z`. You can also clear the output from the previous command using `C-c C-o`. For example, the output of the `'ifconfig'` command is shown below:

```
/tmp $ ifconfig docker0
docker0  Link encap:Ethernet  HWaddr 56:84:7a:fe:97:99
         inet addr:172.17.42.1  Bcast:0.0.0.0
Mask:255.255.0.0
UP BROADCAST MULTICAST  MTU:1500  Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0
carrier:0
collisions:0 txqueuelen:0
RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

/tmp $
```

After executing `'C-c C-o'`, the previous command output gets cleared as shown below:

```
/tmp $ ifconfig docker0
*** output flushed ***
/tmp $
```

You can move the cursor up to the previously executed commands using `C-c C-p` and move down using `C-c C-n`. You can also cycle backwards and forwards through the command history using the `M-p` and `M-n` shortcuts, respectively. The `C-c C-r` shortcut moves to the first line of the output to the top of the window. You can move the cursor to the bottom of the window using `C-c C-e`. The `C-r` key binding allows you to search for a previously typed command from history.

The default shell is Bash, but you can change it to use other shells like `ksh`, `csh` or `zsh`. You can open another shell buffer using `C-u M-x shell`. The new shell buffer will be called `*shell*<2>`. You can, of course, rename this buffer.

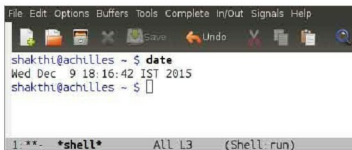


Figure 1: Shell

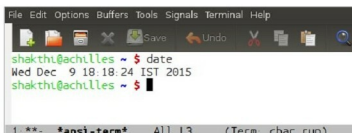


Figure 2: ansi-term

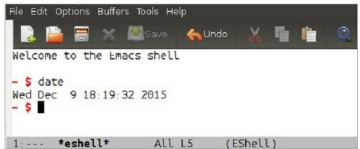


Figure 3: eshell

You can thus open multiple shells to work inside GNU Emacs.

GNU Emacs also has a terminal emulator that you can invoke using `M-x ansi-term`. You can start a Bash session with this command to get the actual colours that you see in a terminal session. This is like a fallback shell if you do not want to use `'M-x shell'`. Eshell is the Emacs built-in shell written completely in Emacs Lisp (Elisp). You can start it using `M-x eshell`. The advantage of using this is that you can extend it, write your own customised Elisp functions and use it similar to shell scripting.

Screenshots of shell, ansi-term and eshells in GNU Emacs are shown in Figures 1, 2 and 3, respectively.

Spell-check commands

You can check the spelling of a word by placing the cursor on it and using the `M-$` shortcut. For example, for the word `'hte'`, GNU Emacs provides the following spelling options:

```
(0) hate (1) HT (2) ht (3) GTE (4) the (5) He (6) Te (7) he (8) Hts
(9) hie (<): hoe (<): hue (<): Rte (=) Ste (>) Ute (@) Bte (B) rte
```

It also lists some options in the minibuffer:

`C-h` or `?` for more options; `SPC` to leave unchanged `Character` to replace word

On pressing the number 4, the word is replaced with the correct spelling. If the spelling is already correct, then GNU Emacs will tell you that the word is correct. You can also spell-check a region using `M-x ispell-region` and a buffer using `M-x ispell-buffer`. If you would like to stop the spell-checker while it is running, you can use `M-x ispell-kill-ispell`. There is a flyspell mode that can check your spelling as you type. You can enable it using `M-x flyspell-mode`. If you want this mode only in a buffer, you can use `M-x flyspell-buffer`.

Word abbreviation

GNU Emacs can complete words for you. If you type `'ba'` and then hit `M-/`, then GNU Emacs will try to complete the word for you. If you continue to use `M-/`, it will cycle through the various options such as `'backwards'`, `'ball'`, `'Bash'`, `'bar'`, `'back'`, `'based'`, etc. To enter into the abbreviation mode, you need to use `M-x abbrev-mode` followed by the `Enter` key.

Continued on Page...81